

constexpr reflexpr

- Document number: **P0953R2**, ISO/IEC JTC1 SC22 WG21
- Date: 2019-11-19
- Authors: Matúš Chochlík <chochlik@gmail.com>, Axel Naumann <axel@cern.ch>, and David Sankel <dsankel@bloomberg.net>
- Audience: SG7 Reflection

Contents

- [Abstract](#)
- [Changes](#)
- [Introduction](#)
 - [From TMP-reflexpr to CXP-reflexpr](#)
 - [Mix compile-time reflection with runtime values](#)
- [Supporting language constructs](#)
 - [constexpr for](#)
 - [constexpr-time allocators](#)
 - [unreflexpr](#)
- [Library considerations](#)
 - [Typeful reflection](#)
 - [References vs. pointers](#)
 - [Downcasting](#)
 - [type traits](#)
- [Type-erased, by-value objects vs. pointers](#)
- [Type-erased, by-value objects vs. monotype](#)
- [Datatypes and Operations](#)
 - [Classes](#)
 - [reflexpr](#)
 - [unreflexpr](#)
- [Conclusion](#)
- [Acknowledgments](#)

Abstract

Before

```
template <typename T>
T min(const T& a, const T& b) {
    using MetaT = reflexpr(T);
    log() << "min<"
        << get_display_name_v<MetaT>
        << ">(" << a << ", " << b << ") = ";
    T result = a < b ? a : b;
    log() << result << std::endl;
    return result;
}
```

After

```
template <typename T>
T min(const T& a, const T& b) {
    constexpr reflect::Type metaT = reflexpr(T);
    log() << "min<"
        << metaT.get_display_name()
        << ">(" << a << ", " << b << ") = ";
    T result = a < b ? a : b;
    log() << result << std::endl;
    return result;
}
```

The reflection TS working draft ([N4766](#)) provides facilities for static reflection that are based on the template metaprogramming paradigm. Recently, however, language features have been proposed that would enable a

more natural syntax for metaprogramming through use of constexpr facilities (See: [P0598](#), [P0633](#), [P0712](#), and [P0784](#)). This paper explores the impact of these language facilities on reflexpr and considers what a natural-syntax-based reflection library would look like.

Changes

- P0953R2 reworked the document to use type-erased, by-value objects per SG7 consensus at the 2018 San Diego meeting. The poll to use “type-erased” by-value objects was SF:3, F:7, N:2, A:1, SA:2.
- P0953R1 introduces a new section, [Library Design Alternative], which, based on feedback from SG7, presents an approach using type-erased, by-value objects. Second, typename was introduced a disambiguator for unreflexpr to address ambiguity in some contexts. Third, a discussion was added to explain the motivation of using pointers instead of references. Finally, the reflect namespace is suggested for placement of type_traits functions.

Introduction

From TMP-reflexpr to CXP-reflexpr

Template-metaprogramming-based reflexpr (TMP-reflexpr), which is succinctly described in [P0578](#), includes a reflexpr operator that, when applied to C++ syntax, produces a *type* that encodes “meta” information about that syntax. For example, when reflexpr is applied to a type, the result can be used to query that type’s name and, in the case of a class, list its member variables.

Consider the following implementation of a function dump which outputs the name and public data members of an arbitrary type. Note that iteration of the data members involves use of an auxiliary function and variadic templates.

```
template <typename T>
void dumpDataMembers(); // implemented below

template <typename T>
void dump()
    // Output the name and data members of a record (class, struct, union).
    // For example:
    //..
    // struct S {
    //     std::string m_s;
    //     int m_i;
    // };
    //
    // int main {
    //     dump<S>(); // name: S
    //              // members:
    //              //     std::string m_s
    //              //     int m_i
    // }
    //..

{
    using MetaT = reflexpr(T);
    std::cout << "name: " << get_display_name_v<MetaT> << std::endl;
    std::cout << "members:" << std::endl;

    using DataMemberObjectSequence = get_public_data_members_t<MetaT>;
```

```

    dumpDataMembers<unpack_sequence<std::tuple, DataMemberObjectSequence>>>();
}

template <>
void dumpDataMembers<std::tuple<>>>() { }
    // base case does nothing

template <typename DataMember, typename... DataMembers>
void dumpDataMembers<std::tuple<DataMember, DataMembers...>>()
{
    // Output information about the first data member and recurse for the
    // subsequent data members.
    std::cout
        << " " << get_display_name_v<get_type_t<DataMember>>>
        << " " << get_display_name_v<DataMember>
        << std::endl;
    dumpDataMembers<std::tuple<DataMembers...>>();
}

```

While the above code can be simplified somewhat by use of a template metaprogramming library such as Boost.MPL, this snippet is fairly representative of the complexity and, for most C++ developers, unfamiliarity of the required constructs.

Louis Dionne's Boost.Hana library provides an alternative approach whereby values with specially designed types allow one to write code that has much of the appearance of normal C++ code, but actually accomplishes metaprogramming (See [P0425](#)). Instead of having to write a separate function to accomplish iteration, one could use `hana::for_each` from within the dump function.

```

template <typename T>
void dump() {
    constexpr auto metaT = reflexpr(T);
    std::cout << "name: " << metaT.get_display_name() << std::endl;
    std::cout << "members:" << std::endl;
    hana::for_each(metaT.get_public_data_members(), [&](auto dataMember) {
        std::cout
            << " " << dataMember.get_type().get_display_name()
            << " " << dataMember.get_display_name()
            << std::endl;
    });
}

```

While this mitigates much of the syntactic complexity of template metaprogramming, the types involved are opaque (they cannot be named) and special library features are required to accomplish tasks such as iteration.

Constexpr-based reflexpr (CXP-reflexpr), the subject of this paper, operates in the Hana-style: instead of reflexpr producing a *type*, it produces a *value* that encodes meta information. Unlike Hana-style, however, this value has a non-opaque type and we normally do not need special library features to work with it.

The following snippet illustrates our original example using CXP-reflexpr.

```

template <typename T>
void dump() {
    constexpr reflect::Record metaT = reflexpr(T);
    std::cout << "name: " << metaT.get_display_name() << std::endl;
    std::cout << "members:" << std::endl;
    for(const RecordMember member : metaT.get_public_data_members())
        std::cout
            << " " << member.get_type().get_display_name()

```

```

        << " " << member.get_name() << std::endl;
    }

```

Mix compile-time reflection with runtime values

In the above example, we didn't have a need to mix a runtime value with compile-time meta-information. While this is doable with TMP-reflexpr, additional language facilities are required to accomplish this with CXP-reflexpr.

Consider a function dumpJson that outputs an arbitrary class or struct as a JSON string. Note the use of two new language constructs: constexpr for, and unreflexpr.

```

void dumpJson(const std::string &s);
    // Output 's' with double-quotes and escaped special characters.

void dumpJson(const int &i);
    // Output 'i' as an int.

template <typename T>
void dumpJson(const T &t) {
    std::cout << "{ ";
    constexpr reflect::Class metaT = reflexpr(T);
    bool saw_prev = false;
    constexpr for(const RecordMember member : metaT.get_public_data_members())
    {
        if(saw_prev) {
            std::cout << ", ";
        }

        std::cout << member.get_name() << "=";

        constexpr reflect::Constant pointerToMember = member.get_pointer();
        dumpJson(t.*unreflexpr(pointerToMember));

        saw_prev = true;
    }
    std::cout << " }";
}

struct S {
    std::string m_s;
    int m_i;
};

int main {
    dumpJson( S{"hello", 33} ); // outputs: { m_s="hello", m_i=33 }
}

```

The recursive call warrants some additional explanation

```
dumpJson(t.*unreflexpr(pointerToMember));
```

- pointerToMember is “meta” information about a pointer to a member. In the above example, it refers to &S:m_s in the first iteration of the loop and &S:m_i in the second iteration.
- The unreflexpr operator converts this “meta” information about the pointer to member to the pointer to member itself.
- t.* is the syntax for accessing a pointer to member.

- `dumpJson` is the recursive call.

The first iteration is decomposed like this:

```
dumpJson(t.*unreflexpr(pointerToMember));
      ↓
dumpJson(t.*&S::m_s);
      ↓
dumpJson(t.m_s);
```

The second iteration is similar:

```
dumpJson(t.*unreflexpr(pointerToMember));
      ↓
dumpJson(t.*&S::m_i);
      ↓
dumpJson(t.m_i);
```

Supporting language constructs

`constexpr for`

This construct essentially unrolls a for loop at compile-time, allowing for the body of the loop to manifest different types at different iterations. This was originally proposed by Andrew Sutton in [P0589](#) for Hana-style programming, but the unrolling semantics work for our purposes as well.

While this language construct isn't strictly necessary for CXP-reflexpr, we think it greatly simplifies code that would otherwise require complex library facilities.

`constexpr-time allocators`

`constexpr-time` allocators as described in [P0784](#) (also seen in a more preliminary form in [P0597](#)) allow us to make use of `std::vector`, among other data types, at compile time. While these constructs aren't strictly necessary (resizable arrays based on fixed buffer sizes has been demonstrated by Ben Deane and Jason Turner in [P0810](#)), they make the data structures used at compile time more uniform with those at runtime.

`unreflexpr`

With TMP-reflexpr, types are easily extracted because we are already in a type context.

```
// 'foo' has a type that is the same as the first field of 'S'.
get_reflected_type_t<
    get_type_t<
        get_element_t<
            0,
            get_public_data_members_t<reflexpr(S)>>>> foo;
```

With CXP-reflexpr, on the other hand, once we have a Type object, there isn't a sufficient language facility for going back into type processing

```
constexpr Type t = reflexpr(S).get_public_data_members()[0].get_type();
```

```
// unreflexpr is required to create 'foo' with the type that 't' refers to.
unreflexpr(t) foo;
```

One might think `decltype` would work, but the `Type` returned by `reflexpr(X)` is generic, and has no specialized member types for `X`; there is nothing `X`-specific to `decltype()` on.

Therefore, the one language-level feature that is critical for feature parity with TMP-`reflexpr` is `unreflexpr` support. It is required for both types and compile-time values.

Library considerations

A `constexpr reflexpr` facility has several library-level implications and design questions. These are addressed in this section.

Typeful reflection

Some of the initial sketches (and recently [P1240](#) by Andrew Sutton, Faisal Vali, and Daveed Vandevoorde) for `constexpr`-based reflection have the `reflexpr` operation produce values that are always of the same type. While this has some advantages for implementers and is certainly simpler than each value having its own unique type, we feel that making some use of types will encourage programming that is easier to read and reason about.

- Typeful reflection enables the use of overloading in library development. Consider the simplicity of the following overloaded function when compared to a single function implemented with `constexpr if`.

```
void outputMetaInformation(reflect::Union);
void outputMetaInformation(reflect::Class);
void outputMetaInformation(reflect::Enum);
void outputMetaInformation(reflect::Type);
void outputMetaInformation(reflect::TypeAlias);
void outputMetaInformation(reflect::Namespace);
void outputMetaInformation(reflect::NamespaceAlias);
void outputMetaInformation(reflect::RecordMember);
void outputMetaInformation(reflect::Variable);
void outputMetaInformation(reflect::Enumerator);
```

- Typeful reflection is similar to the already accepted and sufficiently motivated concepts-based TMP `reflexpr`. Types are to values what concepts are to types.
- Using untyped reflection has many of the same problem as treating all memory as `void*`. Reasonably-strong types help with organization and the long-term maintenance of programs.

We also considered using `std::variant` or variant-like classes as an alternative to reference semantics. Due to the complexity of visitation, the resulting code ended up complex looking, especially for those newer to the language. If a language-level variant with pattern matching support were to be incorporated into C++, then this might be worth revisiting.

References vs. pointers

We were initially tempted to use references to provide a value-semantic experience. Unfortunately, the inability to put a reference directly in a `std::vector` hinders usability. This design choice would require use of the often confusing `std::reference_wrapper` template in several places.

Downcasting

While reflecting syntax will produce the most-specific type available, the need to go from a general type to a specific type remains. For example, the return type of `Record::get_public_member_types` is `std::vector<Type>`. A user may want to “downcast” one of these types into a `Class`, for example.

For this, we provide two cast-related operations in our Object class:

```
class Object {
public:
    // ...

    template<typename T>
    constexpr T get_as() const;
    // Return the specified view, but constexpr-throw (aka diagnose) in
    // case of invalid accesses.

    template<typename T>
    constexpr bool is_a() const;
    // Returns whether or not this object can be viewed as the specified
    // 'T'.
};
```

These functions allow a user to check if the object can be downcast and to actually perform the operation.

type_traits

The C++ standard library includes several metafunctions that aid in metaprogramming in the <type_traits> header. `std::add_const_t` is one such example. While these facilities can be used in the CXP-reflexpr paradigm, it is awkward:

```
Type p = /*...*/;
Type constP = reflexpr(std::add_const_t<unreflexpr(p)>);
```

We propose that each of these existing metaprogramming functions get a CXP-reflexpr-styled equivalent in the `reflect` namespace.

```
namespace std::reflect {
    constexpr reflect::Type add_const(reflect::Type t)
    {
        // Not necessarily implemented in this way.
        return reflexpr(std::add_const_t<unreflexpr(t)>);
    }
}
```

Type-erased, by-value objects vs. pointers

While the use of pointers and an inheritance hierarchy for user syntax has benefits, there are two principle problems with this approach. First, as hinted at in [P0993r0](#), the storage and linkage of the pointed-to value raises some implementability concerns that, while likely possible to mitigate, significantly increase the complexity of the approach. Second, we have observed a preference from the committee to use value semantics instead of pointer semantics whenever possible.

Type-erased, by-value objects vs. monotype

[P0993r0](#) advocated for an approach where `reflexpr` would always return values of type `meta::object`. This forces use of concepts in the case of overloading. The following snippet shows the changed signatures of `outputMetaInformation` as described in [Typeful reflection](#):

```
template<reflect::Union u>
void outputMetaInformation();
```

```
template<reflect::Class c>
void outputMetaInformation();
```

Note that `reflect::Union` and `reflect::Class` are concepts and not types. `u` and `c`, in this approach, both have type `reflect::object`.

There are three principle drawbacks of this approach. First overloading must always use values passed by a template parameter, even when it would not otherwise be necessary. This may substantially discourage creation and maintenance of reflection code by the large number of C++ developers that would not consider themselves experts in the language. Second, requiring concepts to do basic things goes against the desire to make metaprogramming look just like normal programming. Third, the lack of built-in types may result in the proliferation of programs which use `reflect::object` without concepts and create a maintenance burden.

Type-erased, by-value objects are preferred. This is like the ‘pointer’ approach in that there is a hierarchy of types, but conversion operators are used instead of casting to base classes. For example, the `RecordMember` type would be,

```
class RecordMember
{
public:
    constexpr bool is_public() const;
    constexpr bool is_protected() const;
    constexpr bool is_private() const;

    constexpr Record get_type() const;
    constexpr operator Named() const;
};
```

. This approach provides the benefits of typeful programming and those of the monotype approach.

Consider the difference between the `get_public_data_members` function of `Record` with the monotype style,

```
class Record
{
public:
    //...
    constexpr std::vector<meta::object> get_public_data_members() const;
};
```

// Type provides little information here

```
std::vector<meta::object> members
    = reflexpr(SomeType).get_public_data_members();
```

// Alternatively, assuming some kind of terse concepts syntax.

```
std::vector<meta::RecordMember {}> members
    = reflexpr(SomeType).get_public_data_members();
```

, and the type-erased, by-value style,

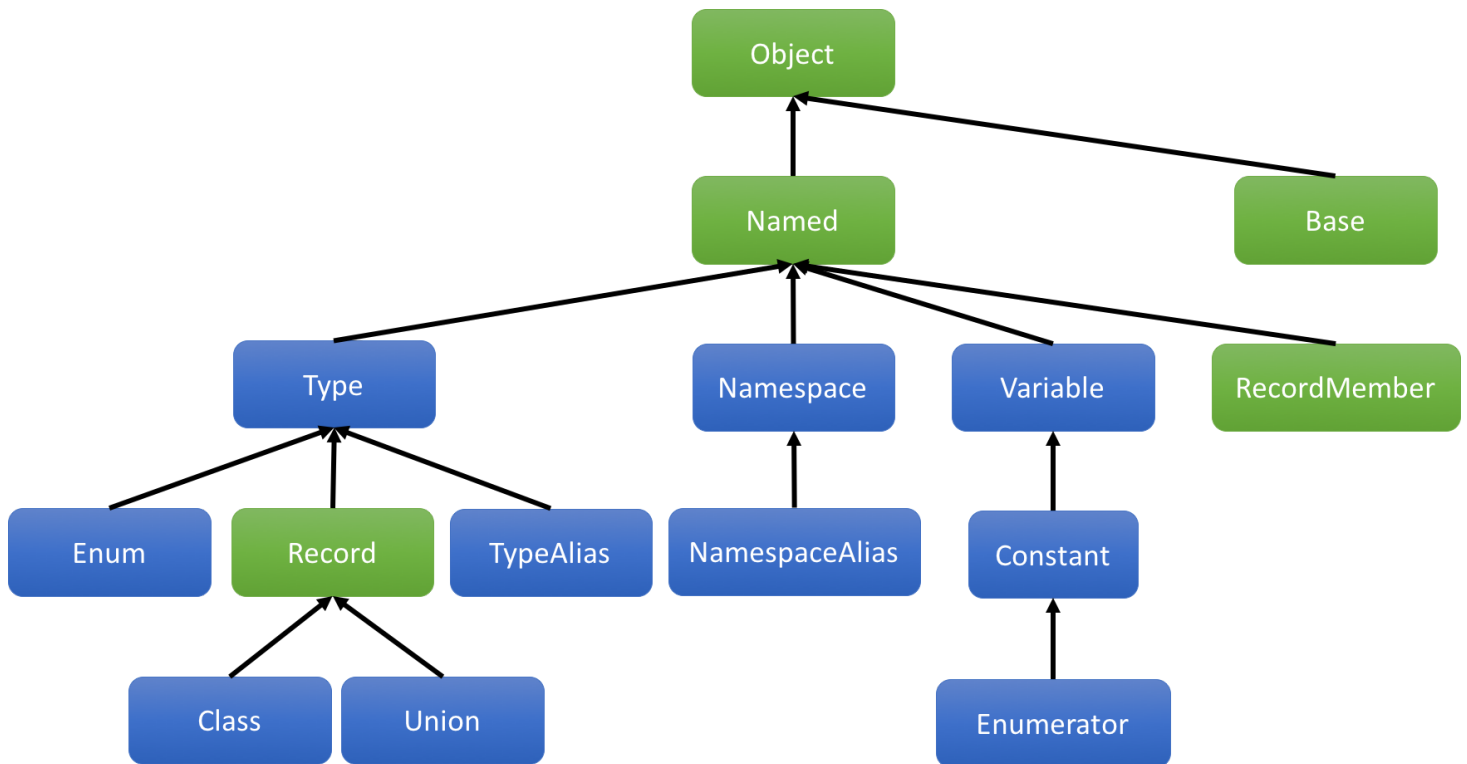
```
class Record
{
    //...
    constexpr std::vector<RecordMember> get_public_data_members() const;
};
```

```
std::vector<RecordMember> members
    = reflexpr(SomeType).get_public_data_members();
```

. The latter appears at a glance to be typical C++ code.

Datatypes and Operations

CXP-reflexpr provides a rich type hierarchy representing the various attributes that can be reflected. The following diagram illustrates this hierarchy.



Arrows go from logically derived classes to logically base classes. The blue classes are those that reflexpr directly produces values of. The green classes are those that are intermediate or indirectly available from the other classes.

Classes

What follows is a short synopsis of the hierarchy described above.

```
namespace reflect {  
  
class Object {  
public:  
    constexpr std::source_location get_source_location() const;  
    constexpr std::string get_source_file_name() const;  
  
    template<typename T>  
    constexpr T get_as() const;  
  
    template<typename T>  
    constexpr bool is_a() const;  
};  
  
class Named {  
public:  
    // Object operations
```

```

// ...
constexpr operator Object() const;

constexpr bool is_anonymous() const;
constexpr std::string get_name() const;
constexpr std::string get_display_name() const;
};

class Type {
public:
    // Named operations
    // ...
    constexpr operator Named() const;
};

class Record {
public:
    // Type operations
    // ...
    constexpr operator Type() const;

    constexpr std::vector<RecordMember> get_public_data_members() const;
    constexpr std::vector<RecordMember> get_accessible_data_members() const;
    constexpr std::vector<RecordMember> get_data_members() const;

    constexpr std::vector<Type> get_public_member_types() const;
    constexpr std::vector<Type> get_accessible_member_types() const;
    constexpr std::vector<Type> get_member_types() const;
};

class Union {
public:
    // Record operations
    // ...
    constexpr operator Record() const;
};

class Class {
public:
    // Record operations
    // ...
    constexpr operator Record() const;

    constexpr bool is_struct() const;
    constexpr bool is_class() const;

    constexpr std::vector<Base> get_public_bases() const;
    constexpr std::vector<Base> get_accessible_bases() const;
    constexpr std::vector<Base> get_bases() const;

    constexpr bool is_final() const;
};

class Enum {
public:
    // Type operations
    // ...
    constexpr operator Type() const;
};

```

```

    constexpr bool is_scoped_enum() const;
    constexpr std::vector<Enumerator> get_enumerators() const;
};

class TypeAlias {
public:
    // Type operations
    // ...
    constexpr operator Type() const;

    constexpr Type get_aliased() const;
};

class Variable {
public:
    // Named operations
    // ...
    constexpr operator Named() const;

    constexpr bool is_constexpr() const;
    constexpr bool is_static() const;

    constexpr Constant get_pointer() const;
    constexpr Type get_type() const;
};

class Base {
public:
    // Object operations
    // ...
    constexpr operator Object() const;

    constexpr Class get_class() const;
    constexpr bool is_virtual() const;
    constexpr bool is_public() const;
    constexpr bool is_protected() const;
    constexpr bool is_private() const;
};

class Namespace {
public:
    // Named operations
    // ...
    constexpr operator Named() const;

    constexpr bool is_global() const;
    constexpr bool is_inline() const;
};

class NamespaceAlias {
public:
    // Namespace operations
    // ...
    constexpr operator Namespace() const;

    constexpr Namespace get_aliased() const;
};

```

```
};

class RecordMember {
public:
    // Named operations
    // ...
    constexpr operator Named() const;

    constexpr bool is_public() const;
    constexpr bool is_protected() const;
    constexpr bool is_private() const;

    constexpr Record get_type() const;
};

class Constant {
public:
    // Variable operations
    // ...
    constexpr operator Variable() const;
};

class Enumerator {
public:
    // Constant operations
    // ...
    constexpr operator Constant() const;
};

} // namespace reflect
```

reflexpr

reflexpr(texp) returns values of types according to the first rule that applies:

- If texp is a type alias, a TypeAlias is returned.
- If texp is an enum, a Enum is returned.
- If texp is a class, a Class is returned.
- If texp is a union, a Union is returned.
- If texp is any other type, a Type is returned.
- If texp is a namespace alias, a NamespaceAlias is returned.
- If texp is a namespace, a Namespace is returned.
- If texp is an enumerator, a Enumerator is returned.
- If texp is a compile-time constant, a Constant is returned.
- If texp is a variable, a Variable is returned.

unreflexpr

unreflexpr(meta) produces the entities according to the following rules:

- If meta has type Type, the result is the type that meta reflects.
- If meta has type Constant, the result is the value that meta reflects. In an otherwise ambiguous context, meta must have type Constant.

unreflexpr typename(meta) produces the entities according to the following rules:

- `meta` must have type `Type`, the result is the type that `meta` reflects.

Conclusion

This paper introduces a facility that allows for compile-time reflection using a natural, `constexpr`-styled programming by taking advantages of new `constexpr` language features on the horizon. The result is a much simplified interface that makes both reflection and metaprogramming more accessible and maintainable.

Acknowledgments

None of this would be possible without the continued, pioneering work of Daveed Vandevoorde, Louis Dionne, and Andrew Sutton in improving the experience of `constexpr` programming.